

APPLICATION NOTE

Linux 下 ATBM BLE DEMO 配网使用方法



ATBM606X

1x1

802.11b/g/n/ax

Wi-Fi/BLE 芯片

Table of contents

1	BLE SMART DEMO 使用方法	
2	BLE DEMO 修改	错误！未定义书签
3	BLE 配网	
4	BLE 配网程序代码初始化与配网数据交互流程	
5	移植自定义应用代码与退出 BLE 线程	
6	BLE RF 测试指令	



作者	版本	说明
Yuzhihuang	V1.0	init
Fuzhenzhou	V1.1	增加 BLE 配网代码配置说明

1 BLE SMART DEMO 使用方法

ble_smart_cfg 源码在驱动根目录的 ble_host\user_app\ble_smart_cfg 目录下
完整的编译完驱动会生成如下文件：

```
ubuntu_sz_2ndfuzhenzhou~/6062/atbm_wifi6_40M/driver_install\ $ls
ATBM606x_wifi_usb.ko  atbm_cli  atbm_tool  cfg80211.ko  nimble linux t
```

在 Linux 系统里面直接执行：

```
insmod cfg80211.ko
```

```
insmod ATBM606x_wifi_usb.ko wifi_bt_comb=1 //1 是 WiFi+BLE 驱动驱动 0 是完整版驱动
```

```
./nimble_linux & //BLE 运行
```

2 BLE 配网

连接脚本需要在根目录\ble_host\user_app\目录下把 connect_ap.sh 复制到 linux 系统 tmp 目录下执行。

脚本代码如下：

```
connect_ap.sh
19 else
20     echo -e "
21     ctrl_interface=/tmp/wpa_supplicant\n
22     ap_scan=1\n
23     network={\n
24     \tssid=\"$1\"\n
25     \tpsk=\"$2\"\n
26     \tscan_ssid=1\n
27     }\n
28     " > /tmp/wpa_supplicant.conf
29 fi
30 }
31
32
33
34
35
36
37 connect_ap()
38 {
39
40     write_wpa_conf $1 $2
41
42     wpa_supplicant -Dnl80211 -iwlan0 -c /tmp/wpa_supplicant.conf -B
43     udhcpc -iwlan0 &
44 }
45
46
47 connect_ap $1 $2
48
49
```

通过BLE配网流程拿到SSID和PWD写成
wpa_supplicant.conf文件再启动连接

ble_smart_demo.c

Symbol Name (Alt+L)

- u16
- s32
- u32
- s64
- u64
- MAX_SYNC_EVE
- ATBM_SMT_STA
- ATBM_SMT_STA
- cfg_done_conne
- connect_ap
- status_async
 - type
 - driver_mode
 - list_empty
 - event_buffer
- at_cmd_direct_i
 - len
 - cmd
- ble_smt
- gCmdStr
- if 0
- endif
- if 0
- endif
- if 0
- endif
- cmd_system

```

238:
239:     printf("wpa_success %d\n", wpa_success);
240:     return wpa_success;
241:
242: } « end smt_check_wpa_status »
243:
244: void connect_wifi_ap(u8* ssid, u8 ssidLen, u8* pwd, u8 pwdLen)
245: {
246:     u8 ssidStr[33] = { 0 };
247:     u8 pwdStr[65] = { 0 };
248:     int sret, i;
249:     int conn_ap_len;
250:     if (ssidLen > 32 || pwdLen > 64) {
251:         fprintf(stdout, "ssidLen(%d) pwdLen(%d) error!\n", ssidLen, pwdLen);
252:         return;
253:     }
254:
255:     memcpy(ssidStr, ssid, ssidLen);
256:     memcpy(pwdStr, pwd, pwdLen);
257:     conn_ap_len = strlen(connect_ap);
258:     if(conn_ap_len){
259:         if(pwdLen)
260:             sprintf(gCmdStr, "%s %s %s", connect_ap, ssid, pwd);
261:         else
262:             sprintf(gCmdStr, "%s %s", connect_ap, ssid);
263:     }else{
264:         if(pwdLen)
265:             sprintf(gCmdStr, "/tmp/connect_ap.sh %s %s", ssid, pwd);
266:         else
267:             sprintf(gCmdStr, "/tmp/connect_ap.sh %s", ssid);
268:     }
269:     printf("start CMD:%s \n", gCmdStr);
270:     system(gCmdStr);
271:     // cmd_system(gCmdStr, NULL, 0);
272:     #if 0
273:     sprintf(gCmdStr, "ifconfig wlan0 down");
274: 
```

微信直接搜索小程序：ATBM 配网



WeChat



蓝牙配网

快速配网

AP配网

点击蓝牙配网



输入要连接的路由器的 ssid & password 点击 WIFI 连接

3 BLE 配网程序代码初始化与配网数据交互流程

1.主程序入口

目录 ble_host\nimble_v42\apps\main.c



2 BLE 初始线程初始化



3.BLE 使用 API

BLE 广播主要对 ble_hs_adv_fields 初始化，最长 31 个字节。如果需要 2 个广播包就需要配置扫描回应包。

Api:

ble_gap_adv_set_fields(const struct ble_hs_adv_fields *adv_fields) 配置广播包

ble_gap_adv_rsp_set_fields(const struct ble_hs_adv_fields *rsp_fields) 配置扫描回应包

连接成功后通讯字节长度 MTU 设置 api

int ble_att_set_preferred_mtu(uint16_t mtu)

Int ble_gap_adv_start(uint8_t own_addr_type, const ble_addr_t *direct_addr,

int32_t duration_ms,

const struct ble_gap_adv_params *adv_params,

ble_gap_event_fn *cb, void *cb_arg) //启动广播

int ble_gap_adv_stop(void) //停止广播

Int ble_gap_adv_active(void) //返回为 1 时处于广播状态

Int ble_gap_set_default_tx_power(int8_t dbm) //dbm 范围值(0~15)对于广播而言都必须要在设置广播参数之前设置功率，否则不起作用。

Int ble_hs_id_addr(uint8_t id_addr_type, const uint8_t **out_id_addr, int *out_is_nrpa) //获取到蓝牙默认 mac 地址，小端格式存储

Int ble_gap_terminate(uint16_t conn_handle, uint8_t hci_reason) //断开手机连接

Int ble_gap_update_params(uint16_t conn_handle, const struct ble_gap_upd_params *params) //申请更改连接间隔

BLE 线程初始化入口:

设置发射功率与获取 BLE 的默认 mac 地址, BLE MAC 地址默认是 WIFI MAC 地址最后一位+1



```

800:     smt_demo_end_indicate((u8*)&smt_ind);
801: }
802:
803: static int ble_smart_cfg_task(void* param)
804: {
805:     int rc;
806:     int wifi_connected_status = 0;
807:     uint8_t atbm_ble_addr[6];
808:     smart_cfg_start = 1;
809:
810:     ble_npl_time_t wifi_connected_start = 0;
811:     ble_npl_time_t wifi_connected_cur = 0;
812:
813:     ble_npl_sem_init(&hci_smt_sem, 0);
814:
815:     // rc = ble_hs_id_infer_auto(0, &blesmt_addr_type);
816:     // assert(rc == 0);
817:     blesmt_addr_type = BLE_OWN_ADDR_PUBLIC;
818:     atbm_ble_is_quit=0;
819:     ble_gap_set_default_tx_power(0);
820:     /* Begin advertising */
821:     blesmt_advertise();
822:     get_ble_addr(atbm_ble_addr);
823:     //iot_printf("pub addr %02hhx:%02hhx:%02hhx:%02hhx:%02hhx\n", atbm_ble_addr[5], atbm_ble_addr[4], atbm_ble_addr[3], atbm_ble_addr[2], atbm_ble_addr[1], atbm_ble_addr[0]);
824: #if ATBM_BLE_SMART_APP
825:     while (!atbm_ble_is_quit) {
826:         //iot_printf(" ble_smart_cfg_task wait... step %d\n", ble_smart_info.step);
827:
828:         if (ble_smart_info.step != BLE_SMART_CFG_SSID_PWD_TRANS_END) {
829:             rc = ble_npl_sem_pend(&hci_smt_sem, ble_npl_time_ms_to_ticks32(1000)); //BLE_SMART_CFG_TIMEOUT_MS 1000ms
830:         }
831:         else {
832:             //rc = ble_npl_sem_pend(&hci_smt_sem, ble_npl_time_ms_to_ticks32(1000));
833:             sleep(1);
834:             rc = 0;
835:         }
836:
837:         if (rc == 0) {
838:             if (ble_smart_info.step == 4) {
839:                 iot_printf("ble smart cfg connect ap! ssid:%s pwd:%s\n", ble_smart_info.ssid, ble_smart_info.pwd);
840:                 wifi_connected_start = ble_npl_time_get();
841:                 wifi_connected_cur = ble_npl_time_ticks_to_ms32(wifi_connected_start);
842:                 ble_smart_cfg_indication(BLE_SMART_CFG_SSID_PWD_TRANS_END, &ble_smart_info);
843:                 ble_smart_info.step = BLE_SMART_CFG_SSID_PWD_TRANS_END;
844:                 //break;
845:             }
846:             else if (ble_smart_info.step == BLE_SMART_CFG_SSID_PWD_TRANS_END) {
847:                 //check wifi connect status
848:                 // ble_smart_cfg_indication(BLE_SMART_CFG_STATUS_END, &ble_smart_info);
849:             }
850:         }
851:     }
852: }

```

BLE 广播包类型与描述

AD Type	Value	描述
Flags	0x01	广播出自己蓝牙某些特性
Service UUIDs	0x02~0x07	广播出自己服务的 UUID
Local Name	0x08/0x09	广播出自己的蓝牙名字
TX Power Level	0x0A	广播出自己的射频发射功率
Simple Pairing Option OOB Tags	0x0D~0x0F	广播出安全管理带外标签(本文忽略)
Security Manager TK Value	0x10	广播出带外方式配对绑定时的 TK(本文忽略)
Security Manager OOB Flags	0x11	广播出带外特性标志(本文忽略)
Slave Connection Interval Range	0x12	广播出自己希望的连接参数范围
Service Solicitation	0x14/0x15	广播出自己希望来接自己的主机有特定的服务
Service Data	0x16	服务数据
Manufacturer Specific Data	0xFF	广播出厂商信息(用户可以放自定义数据)

ATBM 广播包结构体:

广播包 31 字节和扫描回应包 31 字节配置结构体都是一样。如下:

```

struct ble_hs_adv_fields {
    /** 0x01 - Flags. */
    uint8_t flags;

    /** 0x02,0x03 - 16-bit service class UUIDs. */
    ble_uuid16_t *uuids16;
    uint8_t num_uuids16;

```

```
unsigned uuids16_is_complete:1;

/** 0x04,0x05 - 32-bit service class UUIDs. */
ble_uuid32_t *uuids32;
uint8_t num_uuids32;
unsigned uuids32_is_complete:1;

/** 0x06,0x07 - 128-bit service class UUIDs. */
ble_uuid128_t *uuids128;
uint8_t num_uuids128;
unsigned uuids128_is_complete:1;

/** 0x08,0x09 - Local name. */
uint8_t *name;
uint8_t name_len;
unsigned name_is_complete:1;

/** 0x0a - Tx power level. */
int8_t tx_pwr_lvl;
unsigned tx_pwr_lvl_is_present:1;

/** 0x0d - Slave connection interval range. */
uint8_t *slave_itvl_range;

/** 0x16 - Service data - 16-bit UUID. */
uint8_t *svc_data_uuid16;
uint8_t svc_data_uuid16_len;

/** 0x17 - Public target address. */
uint8_t *public_tgt_addr;
uint8_t num_public_tgt_addrs;

/** 0x19 - Appearance. */
uint16_t appearance;
unsigned appearance_is_present:1;

/** 0x1a - Advertising interval. */
uint16_t adv_itvl;
unsigned adv_itvl_is_present:1;

/** 0x20 - Service data - 32-bit UUID. */
uint8_t *svc_data_uuid32;
uint8_t svc_data_uuid32_len;
```



```

/** 0x21 - Service data - 128-bit UUID. */
uint8_t *svc_data_uuid128;
uint8_t svc_data_uuid128_len;

/** 0x24 - URI. */
uint8_t *uri;
uint8_t uri_len;

/** 0xff - Manufacturer specific data. */
uint8_t *mfg_data;
uint8_t mfg_data_len;
};

```

4.手机搜到 ble 广播数据显示


AltoBeam
C0:55:44:33:22:C0
NOT BONDED ▲ -72 dBm ↔ 44 ms
Device type: LE only
Advertising type: Legacy
Complete list of 16-bit Service UUIDs:
0xBBB0, 0xBBB1
Complete Local Name: AltoBeam
Manufacturer data (Bluetooth Core 4.1):
Company: Reserved ID <0xABAB> 0xA0
Complete list of 16-bit Service UUIDs:
0xCCC0, 0xDDD0
Complete Local Name: AltoBeam
Manufacturer data (Bluetooth Core 4.1):
Company: Reserved ID <0xABAB> 0xA0

CONNECT

Raw data:
0x0503B0BBB1BB0909416C746F42
65616D04FFABABA00503C0CCD0D
D0909416C746F4265616D04FFABA
BA0

Details:

LEN.	TYPE	VALUE
5	0x03	0xB0BBB1BB
9	0x09	0x416C746F4265616D
4	0xFF	0xABABA0
5	0x03	0xC0CCD0DD
9	0x09	0x416C746F4265616D
4	0xFF	0xABABA0

LEN. - length of EIR packet (Type + Data) in bytes,
TYPE - the data type as in [Generic Access Profile.pdf](#)

CLONE RAW MORE

OK

上图广播包初始化代码如下：

```

#define BLE_USER_DATA_SVC_UUID16 0x181C
#define BLE_ATBM_SMART_DATA 0x2A8A
#define BLE_ATBM_SMART_NOTIFY 0x2A90

#define BLE_ATBM_GAP_SVR 0xBBB0
#define BLE_ATBM_GAP_SVR1 0xBBB1

static const char *device_name = "AltoBeam";//广播名字
static const char *ble_version = "nimble4.2";
static const char *firmware_info = "cronus_1.0";
static const char *software_info = "V1.0.0";
static const char* wifi_conn_ok = "ok";
static const char* wifi_conn_no = "no";

static uint8_t blesmt_addr_type;
uint16_t blesmt_notify_handle;
struct ble_smart_cfg_t ble_smart_info;
int smart_cfg_start = 0;
static struct ble_npl_sem hci_smt_sem;
static uint16_t smt_conn_handle;
pAtbm_thread_t ble_smt_thread = NULL;

static void blesmt_advertise(void)
{
    struct ble_gap_adv_params adv_params;
    struct ble_hs_adv_fields fields;
    ble_uuid16_t uuids16[2];
    uint8_t mfg_data[3] = {0xAB, 0xAB, 0xA0};//自定义广播自己私有数据，
    int rc;
    memset(&fields, 0, sizeof(fields));
    //广播数据
    fields.name = (uint8_t *)device_name;
    fields.name_len = strlen(device_name);
    fields.name_is_complete = 1;//选这 0x09 Complete local name
    // #define GAP_ADTYPE_LOCAL_NAME_SHORT 0x08 //! < Shortened local name
    // #define GAP_ADTYPE_LOCAL_NAME_COMPLETE 0x09 //! < Complete local name

    uuids16[0].u.type = BLE_UUID_TYPE_16;
    uuids16[0].value = BLE_ATBM_GAP_SVR;//声明存在 GAP服务
    uuids16[1].u.type = BLE_UUID_TYPE_16;
    uuids16[1].value = BLE_ATBM_GAP_SVR1;//声明存在 GAP服务
    fields.uuids16 = &uuids16[0];
    fields.num_uuids16 = 2;//声明了几个 uuid
    fields.uuids16_is_complete = 1;//咨询 gap格式是完整还是非完整,上面填的是完整所以选择广播数据
    // #define GAP_ADTYPE_16BIT_MORE 0x02 //! < Service: More 16-bit UUIDs available
    // #define GAP_ADTYPE_16BIT_COMPLETE 0x03 //! < Service: Complete list of 16-bit UUIDs

    //uint8_t public_tgt_addr[6] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06};
    //fields.public_tgt_addr=public_tgt_addr;
    //fields.num_public_tgt_addrs=1;

    fields.mfg_data = mfg_data;//厂商私有协议
    fields.mfg_data_len = 3;//注意必须跟 mfg_data数组大小一样

    rc = ble_gap_adv_set_fields(&fields);//广播包配置
    if (rc != 0) {
        log_printf("error setting advertisement data; rc=%d\n", rc);
        return;
    }
}

```

扫描回应包初始化跟广播包初始化一样。只是换成了 `ble_gap_adv_rsp_set_fields` 进行初始化。代码如下：

```
//以下是扫描回应包配置
struct ble_hs_adv_fields adv_fields;
ble_uuid16_t uuids16_rsp[2];
uint8_t mfg_rsp_data[3] = {0xff, 0xcc, 0xdd}; //自定义广播自己私有数据,
memset(&adv_fields, 0, sizeof(adv_fields));
//广播数据 https://blog.csdn.net/m0_57781768/article/details/127739862
adv_fields.name = (uint8_t*)device_name;
adv_fields.name_len = strlen(device_name);
adv_fields.name_is_complete = 1; //选这 0x09 Complete local name
// #define GAP_ADTYPE_LOCAL_NAME_SHORT 0x08 //! < Shortened local name
// #define GAP_ADTYPE_LOCAL_NAME_COMPLETE 0x09 //! < Complete local name
uuids16_rsp[0].u.type = BLE_UUID_TYPE_16;
uuids16_rsp[0].value = 0xccc0; //声明存在 GAP 服务
uuids16_rsp[1].u.type = BLE_UUID_TYPE_16;
uuids16_rsp[1].value = 0xddd0; //声明存在 GAP 服务
adv_fields.uuids16 = &uuids16_rsp[0];
adv_fields.num_uuids16 = 2; //声明了几个 uuid
adv_fields.uuids16_is_complete = 1; //咨询 gap 格式是完整还是非完整,上面填的是完整所以选择广播数
// #define GAP_ADTYPE_16BIT_MORE 0x02 //! < Service: More 16-bit UUIDs available
// #define GAP_ADTYPE_16BIT_COMPLETE 0x03 //! < Service: Complete list of 16-bit UUIDs

// fields.public_tgt_addr=&mac_addr[0];
// fields.num_public_tgt_addrs=1;
adv_fields.mfg_data = mfg_data;
adv_fields.mfg_data_len = 3; //注意必须跟 mfg_data 数组大小一样
rc = ble_gap_adv_rsp_set_fields(&adv_fields);
if (rc != 0) {
    iot_printf("error setting advertisement rsp data; rc=%d\n", rc);
    return;
}
```

启动广播 api 初始化与 BLE mac 地址设置。

其中 ble_slave_gap_event(struct ble_gap_event *event, void *arg)是连接状态回调函数。

```
rc = ble_att_set_preferred_mtu(32); //设置通讯字节长度
if (rc != 0) {
    iot_printf("ble_att_set_preferred_mtu rc=%d\n", rc);
    return;
}
//广播参数
//不可连接模式;
//定向连接模式;
//无向连接模式; BLE_GAP_CONN_MODE_UND
memset(&adv_params, 0, sizeof(adv_params));
adv_params.conn_mode = BLE_GAP_CONN_MODE_UND;

//不可发现模式;
//有限可发现模式;
//一般可发现模式; BLE_GAP_DISC_MODE_GEN
adv_params.disc_mode = BLE_GAP_DISC_MODE_GEN;
adv_params.itvl_min = 60; //最小广告间隔, 0表示使用默认值, 单位: 0.625ms */
adv_params.itvl_max = 70; //最大广告间隔, 0表示使用默认值, 单位: 0.625ms */
//广播间隔值 20ms-10.24s

blesmt_addr_type = BLE_OWN_ADDR_RANDOM; //BLE_OWN_ADDR_RANDOM;
ble_addr_t addr_t;
uint8_t mac_addr[6] = {0xc0, 0x22, 0x33, 0x44, 0x55, 0xc0}; //小端格式自定义 mac 地址
addr_t.type = BLE_OWN_ADDR_RANDOM;
memcpy(addr_t.val, mac_addr, 6);
// ble_hs_id_set_rnd(mac_addr);
// ble_gap_ext_adv_set_addr(BLE_ADV_INSTANCES, &addr_t);

rc = ble_hs_id_set_rnd(mac_addr);
if (rc != 0) {
    iot_printf("ble_hs_id_set_rnd rc=%d\n", rc);
    return;
}

rc = ble_gap_adv_start(blesmt_addr_type, NULL, BLE_HS_FOREVER,
    &adv_params, ble_slave_gap_event, NULL); //启动广播
if (rc != 0) {
    iot_printf("error ble_gap_adv_start; rc=%d\n", rc);
    return;
}
} ? end blesmt_advertise ?
```

5.连接回调函数与连接成功后请求更新连接参数 ble_gap_update_params 使用

struct ble_gap_upd_params params;

params.itvl_min=60; ///Range: 0x0006 to 0x0C80 Time = N * 1.25 msec 1*1.25=7.5ms

params.itvl_max=70;

params.latency=0; //减少或者设置为 0, 那么每次连接事件中都需要回复 Master 的包, 当然功耗会上升, 但数据发送速度也会提高。

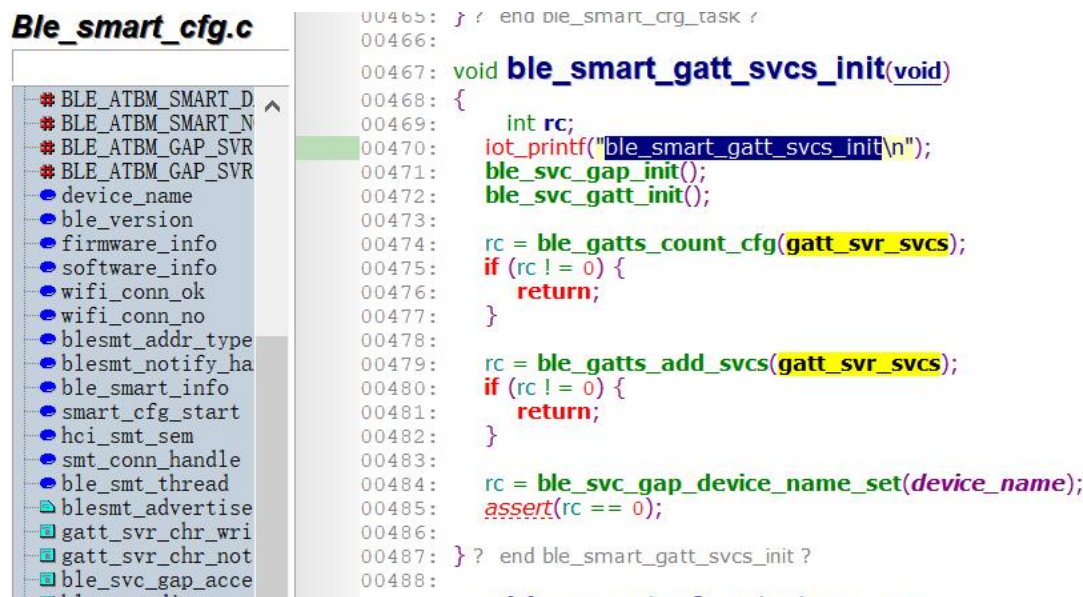
params.supervision_timeout=3000;// 断线超时时间 Supervision timeout for the LE Link.Range: 0x000A to 0x0C80Time = connTimeout* 10 msecTime Range: 100 msec to 32 seconds

ble_gap_update_params(conn_handle, ¶ms);//连接成功后才能请求。同不同意看 master 回应

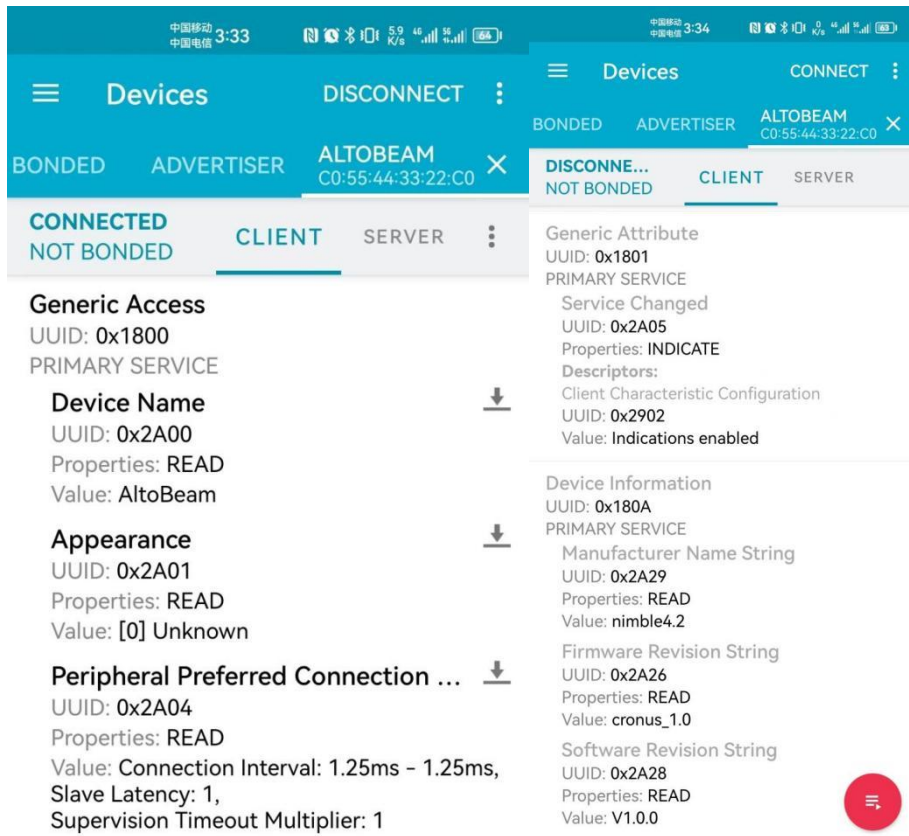


6. 配置参数

7. 通用访问配置文件 (GAP)



手机通过 nrf connect 连接上 ble 后读取到属性表



代码如下进行初始化



事件回调函数接口如下：

Ble_svc_gap.c

```

00177: // : CHU BLE_SVC_GAP_APPEARANCE_WRITE_ACCESS :
00178:
00179: static int
00180: ble_svc_gap_access(uint16_t conn_handle, uint16_t attr_handle,
00181: struct ble_gatt_access_ctxt *ctxt, void *arg)
00182: {
00183:     uint16_t uuid16;
00184:     #if MYNEWT_VAL(BLE_SVC_GAP_CENTRAL_ADDRESS_RESOLUTION) >= 0
00185:     uint8_t central_ar = MYNEWT_VAL(BLE_SVC_GAP_CENTRAL_ADDRESS_RESOLUTION);
00186:     #endif
00187:     #if PPCP_ENABLED
00188:     uint16_t ppcp[4] = {
00189:         htole16(MYNEWT_VAL(BLE_SVC_GAP_PPCP_MIN_CONN_INTERVAL)),
00190:         htole16(MYNEWT_VAL(BLE_SVC_GAP_PPCP_MAX_CONN_INTERVAL)),
00191:         htole16(MYNEWT_VAL(BLE_SVC_GAP_PPCP_SLAVE_LATENCY)),
00192:         htole16(MYNEWT_VAL(BLE_SVC_GAP_PPCP_SUPERVISION_TMO))
00193:     };
00194:     #endif
00195:     int rc;
00196:
00197:     uuid16 = ble_uuid_u16(ctxt->chr->uuid);
00198:     assert(uuid16 != 0);
00199:     iot_printf("ble_svc_gap_access uuid%x\n", uuid16);
00200:     switch (uuid16) {
00201:     case BLE_SVC_GAP_CHR_UUID16_DEVICE_NAME:
00202:         if (ctxt->op == BLE_GATT_ACCESS_OP_READ_CHR) {
00203:             rc = ble_svc_gap_device_name_read_access(ctxt);
00204:         } else if (ctxt->op == BLE_GATT_ACCESS_OP_WRITE_CHR) {
00205:             rc = ble_svc_gap_device_name_write_access(ctxt);
00206:         } else {
00207:             assert(0);
00208:         }
00209:         return rc;
00210:     case BLE_SVC_GAP_CHR_UUID16_APPEARANCE:
00211:         if (ctxt->op == BLE_GATT_ACCESS_OP_READ_CHR) {
00212:

```

GATT 代码初始化

Ble_svc_gatt.c

```

00024: #include "services/gatt/ble_svc_gatt.h"
00025:
00026: static uint16_t ble_svc_gatt_changed_val_handle;
00027: static uint16_t ble_svc_gatt_start_handle;
00028: static uint16_t ble_svc_gatt_end_handle;
00029:
00030: static int
00031: ble_svc_gatt_access(uint16_t conn_handle, uint16_t attr_handle,
00032: struct ble_gatt_access_ctxt *ctxt, void *arg);
00033:
00034: static const struct ble_gatt_svc_def ble_svc_gatt_defs[] = {
00035:     {
00036:         /** Service: GATT */
00037:         .type = BLE_GATT_SVC_TYPE_PRIMARY, // 主要服务项
00038:         .uuid = BLE_UUID16_DECLARE(BLE_GATT_SVC_UUID16),
00039:         .characteristics = (struct ble_gatt_chr_def[]) { {
00040:             .uuid = BLE_UUID16_DECLARE(BLE_SVC_GATT_CHR_SERVICE_CHANGED_UUID16),
00041:             .access_cb = ble_svc_gatt_access, // 接收到数据回调函数
00042:             .val_handle = &ble_svc_gatt_changed_val_handle,
00043:             .flags = BLE_GATT_CHR_F_INDICATE, // 权限
00044:         }, {
00045:             0, /* No more characteristics in this service. */
00046:         } },
00047:     },
00048:     {
00049:         0, /* No more services. */
00050:     },
00051: };
00052:

```

GATT 0x2a05 打开 indication enable 事件回调时进行数据传输示范



GATT 初始化读取图

GATT 初始化入口

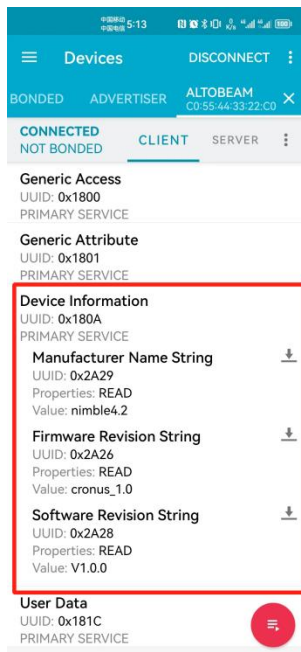
Ble_smart_cfg.c

```

00745: } ? end ble_smart_cfg_task ?
00746:
00747: void ble_smart_gatt_svcs_init(void)
00748: {
00749:     int rc;
00750:     iot_printf("ble_smart_gatt_svcs_init\n");
00751:     ble_svc_gap_init();
00752:     ble_svc_gatt_init();
00753:
00754:     rc = ble_gatts_count_cfg(gatt_svr_svcs); // ble配网初始化
00755:     if (rc != 0) {
00756:         return;
00757:     }
00758:
00759:     rc = ble_gatts_add_svcs(gatt_svr_svcs);
00760:     if (rc != 0) {
00761:         return;
00762:     }
00763:
00764:     rc = ble_svc_gap_device_name_set(device_name);
00765:     assert(rc == 0);
00766:
00767: } ? end ble_smart_gatt_svcs_init ?
00768:

```

Uuid 0x180A Device Information Service: Device Information 手机读取图:



代码配置如图:


```

Ble_smart_cfg.c
00286:    0, /* No more characteristics in this service */
00287: },
00288: },
00289: #endif
00290: {
00291:     /* Service: Device Information 0x1801*/
00292:     .type = BLE_GATT_SVC_TYPE_PRIMARY,
00293:     .uuid = BLE_UUID16_DECLARE(BLE_SVC_DIS_UUID16),
00294:     .characteristics = (struct ble_gatt_chr_def[]) { {
00295:         /* Characteristic: * Manufacturer name */
00296:         .uuid = BLE_UUID16_DECLARE(BLE_SVC_DIS_CHR_UUID16_MANUFACTURER_NAME),
00297:         .access_cb = ble_svc_dis_access, // 回调处理函数
00298:         .flags = BLE_GATT_CHR_F_READ, // 只读权限
00299:     }, {
00300:         /* Characteristic: Model number string */
00301:         .uuid = BLE_UUID16_DECLARE(BLE_SVC_DIS_CHR_UUID16_FIRMWARE_REVISION),
00302:         .access_cb = ble_svc_dis_access, // 回调处理函数
00303:         .flags = BLE_GATT_CHR_F_READ, // 只读权限
00304:     }, {
00305:         /* Characteristic: Model number string */
00306:         .uuid = BLE_UUID16_DECLARE(BLE_SVC_DIS_CHR_UUID16_SOFTWARE_REVISION),
00307:         .access_cb = ble_svc_dis_access, // 回调处理函数
00308:         .flags = BLE_GATT_CHR_F_READ, // 只读权限
00309:     }, {
00310:         0, /* No more characteristics in this service */
00311:     },
00312: },
00313: },

```

Uuid 0x180A Device Information 下面 Characteristic:

Manufacturer name Model number string Model number string 回应

```

Ble_smart_cfg.c
00108: default:
00109:     assert(0);
00110:     return BLE_ATT_ERR_UNLIKELY;
00111: }
00112: return 0;
00113: }
00114: #endif
00115: static const char *ble_version = "nimble4.2";
00116: static const char *firmware_info = "cronus_1.0";
00117: static const char *software_info = "V1.0.0";
00118: static int ble_svc_dis_access(uint16_t conn_handle, uint16_t attr_handle,
00119:                               struct ble_gatt_access_ctxt *ctx, void *arg)
00120: {
00121:     uint16_t uuid = ble_uuid_u16(ctx->chr->uuid);
00122:     const char *info = NULL;
00123:     int rc;
00124:     iot_printf("ble_svc_dis_access uuid%x\n", uuid);
00125:     switch(uuid) {
00126:     case BLE_SVC_DIS_CHR_UUID16_MANUFACTURER_NAME: // 根据对应 uuid 进行回应 read
00127:         info = ble_version;
00128:         break;
00129:     case BLE_SVC_DIS_CHR_UUID16_FIRMWARE_REVISION:
00130:         info = firmware_info;
00131:         break;
00132:     case BLE_SVC_DIS_CHR_UUID16_SOFTWARE_REVISION:
00133:         info = software_info;
00134:         break;
00135:     default:
00136:         assert(0);
00137:         return BLE_ATT_ERR_UNLIKELY;
00138:     }
00139:     if (info != NULL) {
00140:         rc = os_mbuf_append(ctx->om, info, strlen(info));
00141:         return rc == 0 ? 0 : BLE_ATT_ERR_INSUFFICIENT_RES;
00142:     }
00143:     return 0;

```


Ble_svc_gap.c

```

00046:         struct ble_gatt_access_ctxt *ctxt, void *arg);
00047:
00048: static const struct ble_gatt_svc_def ble_svc_gap_defs[] = {
00049: {
00050:     /** Service: GAP. */
00051:     .type = BLE_GATT_SVC_TYPE_PRIMARY, // Primary Service ( 首要服务项 )
00052:     .uuid = BLE_UUID16_DECLARE(BLE_SVC_GAP_UUID16), // Service uuid
00053:     .characteristics = (struct ble_gatt_chr_def[]) { {
00054:         /** Characteristic: Device Name. */
00055:         .uuid = BLE_UUID16_DECLARE(BLE_SVC_GAP_CHR_UUID16_DEVICE_NAME), // Characteristic uuid
00056:         .access_cb = ble_svc_gap_access, // 处理 Characteristic 回调函数
00057:         .flags = BLE_GATT_CHR_F_READ | // 权限只能手机端读
00058:         #if MYNEWT_VAL(BLE_SVC_GAP_DEVICE_NAME_WRITE_PERM) >= 0
00059:             BLE_GATT_CHR_F_WRITE |
00060:             MYNEWT_VAL(BLE_SVC_GAP_DEVICE_NAME_WRITE_PERM) |
00061:         #endif
00062:         0,
00063:     }, {
00064:         /** Characteristic: Appearance. */
00065:         .uuid = BLE_UUID16_DECLARE(BLE_SVC_GAP_CHR_UUID16_APPEARANCE),
00066:         .access_cb = ble_svc_gap_access,
00067:         .flags = BLE_GATT_CHR_F_READ |
00068:         #if MYNEWT_VAL(BLE_SVC_GAP_APPEARANCE_WRITE_PERM) >= 0
00069:             BLE_GATT_CHR_F_WRITE |
00070:             MYNEWT_VAL(BLE_SVC_GAP_APPEARANCE_WRITE_PERM) |
00071:         #endif
00072:         0,
00073:     }, {
00074:         #if PPCP_ENABLED
00075:         /** Characteristic: Peripheral Preferred Connection Parameters. */
00076:         .uuid =
00077:             BLE_UUID16_DECLARE(BLE_SVC_GAP_CHR_UUID16_PERIPH_PREF_CONN_PARAMS),
00078:         .access_cb = ble_svc_gap_access,
00079:         .flags = BLE_GATT_CHR_F_READ,
00080:         #endif
00081:     }, {
00082:         #if MYNEWT_VAL(BLE_SVC_GAP_CENTRAL_ADDRESS_RESOLUTION) >= 0
00083:         /** Characteristic: Central Address Resolution. */
00084:         .uuid = BLE_UUID16_DECLARE(BLE_SVC_GAP_CHR_UUID16_CENTRAL_ADDRESS_RESOLUTION),
00085:         .access_cb = ble_svc_gap_access,

```

BLE 读写可以参考以下事件回调函数实现

```

00177: } } end ble_svc_gap_appearance_write_access ?
00178:
00179: static int
00180: ble_svc_gap_access(uint16_t conn_handle, uint16_t attr_handle,
00181: struct ble_gatt_access_ctxt *ctxt, void *arg)
00182: {
00183:     uint16_t uuid16;
00184:     #if MYNEWT_VAL(BLE_SVC_GAP_CENTRAL_ADDRESS_RESOLUTION) >= 0
00185:     uint8_t central_ar = MYNEWT_VAL(BLE_SVC_GAP_CENTRAL_ADDRESS_RESOLUTION);
00186:     #endif
00187:     #if PPCP_ENABLED
00188:     uint16_t ppcp[4] = {
00189:         htole16(MYNEWT_VAL(BLE_SVC_GAP_PPCP_MIN_CONN_INTERVAL)),
00190:         htole16(MYNEWT_VAL(BLE_SVC_GAP_PPCP_MAX_CONN_INTERVAL)),
00191:         htole16(MYNEWT_VAL(BLE_SVC_GAP_PPCP_SLAVE_LATENCY)),
00192:         htole16(MYNEWT_VAL(BLE_SVC_GAP_PPCP_SUPERVISION_TMO))
00193:     };
00194:     #endif
00195:     int rc;
00196:
00197:     uuid16 = ble_uuid_u16(ctxt->chr->uuid);
00198:     assert(uuid16 != 0);
00199:     iot_printf("ble_svc_gap_access uuid->%x\n", uuid16);
00200:     switch (uuid16) { // 根据 uuid 进行事件响应
00201:     case BLE_SVC_GAP_CHR_UUID16_DEVICE_NAME:
00202:         if (ctxt->op == BLE_GATT_ACCESS_OP_READ_CHR) {
00203:             rc = ble_svc_gap_device_name_read_access(ctxt); // 读属性回调
00204:         } else if (ctxt->op == BLE_GATT_ACCESS_OP_WRITE_CHR) {
00205:             rc = ble_svc_gap_device_name_write_access(ctxt); // 写属性回调
00206:         } else {
00207:             assert(0);
00208:         }
00209:         return rc;
00210:
00211:     case BLE_SVC_GAP_CHR_UUID16_APPEARANCE:
00212:         if (ctxt->op == BLE_GATT_ACCESS_OP_READ_CHR) {
00213:             rc = ble_svc_gap_appearance_read_access(ctxt);
00214:         } else if (ctxt->op == BLE_GATT_ACCESS_OP_WRITE_CHR) {
00215:             rc = ble_svc_gap_appearance_write_access(ctxt);

```

蓝牙配网 gatt 图与代码初始化

Ble_smart_cfg.c

```

00129: return 0;
00130: } ? end ble_svc_dis_access ?
00131:
00132: static int ble_svc_gatt_access(uint16_t conn_handle, uint16_t attr_handle,
00133:                                struct ble_gatt_access_ctxt *ctxt, void *arg)
00134: {
00135:     int rc;
00136:     uint16_t uuid;
00137:     uint8_t data[80] = {0};
00138:     uint16_t len = 0;
00139:
00140:     uuid = ble_uuid_u16(ctxt->chr->uuid);
00141:     iot_printf("ble_svc_gatt_access uuid%x\n", uuid);
00142:     if(uuid == BLE_ATBM_SMART_DATA){ //接收到 app 下发 ssid 与 pwd
00143:         rc = gatt_svr_chr_write(ctxt->om, 0, 80, data, &len);
00144:         if(rc){
00145:             return rc;
00146:         }
00147:         iot_printf("att[%s] Len=%d step %d\n", data, len, ble_smart_info.step);
00148:
00149:         if(((memcmp(data, "recv ok", strlen("recv ok")) == 0) && (ble_smart_info.step == 4))){
00150:             ble_smart_info.step = BLE_SMART_CFG_STATUS_END;
00151:             ble_npl_sem_release(&hci_smt_sem);
00152:         }
00153:
00154:         if(ble_smart_info.step == BLE_SMART_CFG_STATUS_END){
00155:             return 0;
00156:         }
00157:
00158:         switch(ble_smart_info.step)
00159:         {
00160:             case 0:
00161:                 if((memcmp(data, "ssid", strlen("ssid"))) == 0){
00162:                     if(len == 6){
00163:                         ble_smart_info.ssid_len = (data[5] - '0');
00164:                     }
00165:                     else if(len == 7){
00166:                         ble_smart_info.ssid_len = 10*(data[5] - '0') + (data[6] - '0');
00167:                     }
00168:                     iot_printf("ble ssid_len = %d\n", ble_smart_info.ssid_len);
00169:                     if(ble_smart_info.ssid_len > 32){
00170:                         iot_printf("ble ssid_len too big error\n");
00171:                         ble_smart_info.step = BLE_SMART_CFG_STATUS_SSID_LEN_ERR;
00172:                         ble_npl_sem_release(&hci_smt_sem);
00173:                     }

```

BLE 收到 ssid 和 pwd 后进行 notify 发数据手机端的 uuid 0x2a90 中

Ble_smart_cfg.c

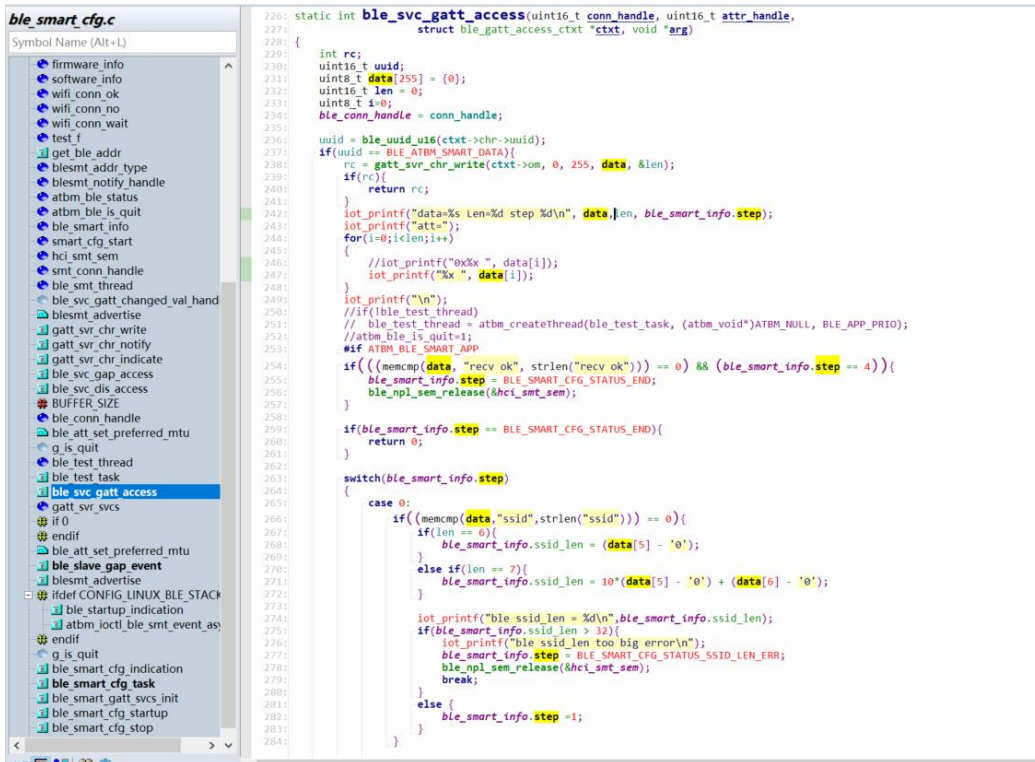
```

00213:     }
00214:     else if((ble_smart_info.pwd_len < 5)&&(ble_smart_info.pwd_len > 0)){
00215:         iot_printf("ble pwd_len too short error\n");
00216:         ble_smart_info.step = BLE_SMART_CFG_STATUS_PWD_LEN_ERR;
00217:         ble_npl_sem_release(&hci_smt_sem);
00218:     }
00219:     else if(ble_smart_info.pwd_len == 0){
00220:         iot_printf("ble pwd_len is empty\n");
00221:         ble_smart_info.step = 4;
00222:     }
00223:     else {
00224:         ble_smart_info.step = 3;
00225:     }
00226: } ? end if (memcmp(data, "pwd", st... ?
00227: break;
00228:
00229: case 3:
00230:     if((ble_smart_info.pwd_cnt + len <= ble_smart_info.pwd_len)){
00231:         memcpy(&ble_smart_info.pwd[ble_smart_info.pwd_cnt], data, len);
00232:         ble_smart_info.pwd_cnt += len;
00233:     }
00234:     else{
00235:         ble_smart_info.step = BLE_SMART_CFG_STATUS_PWD_ERR;
00236:         ble_npl_sem_release(&hci_smt_sem);
00237:         break;
00238:     }
00239:     if(ble_smart_info.pwd_cnt == ble_smart_info.pwd_len){
00240:         ble_smart_info.step = 4;
00241:         iot_printf("ble pwd = %s\n", ble_smart_info.ssid);
00242:     }
00243:     break;
00244:
00245: default:
00246:     break;
00247: } ? end switch ble_smart_info.step ?
00248:
00249: if(ble_smart_info.step == 4){ //收到 ssid 和 pwd 完整后通过 notify 发数据给手机 app 端 uuid 0x2a90 中
00250:     gatt_svr_chr_notify(conn_handle, blesmt_notify_handle, wifi_conn_ok, strlen(wifi_conn_ok)); //成功回复字符 ok
00251: }
00252: else if(ble_smart_info.step >= BLE_SMART_CFG_STATUS_BUSY){
00253:     gatt_svr_chr_notify(conn_handle, blesmt_notify_handle, wifi_conn_no, strlen(wifi_conn_no)); //失败回复字符 no
00254: }
00255: } ? end if uuid==BLE_ATBM_SMART_... ?
00256: return 0;
00257: } ? end ble_svc_gatt_access ?

```

Ble 配网数据交互流程

手机端通过 uuid 0x2a90 write 数据过来，在下面函数中解析。



手机发数据的 log 显示

```

ble_slave_gap_event->type=14
subscribe event; conn_handle=1 attr_handle=23 reason=1 prevn=0 currn=1 pre
i=0
ble_slave_gap_event->type=14
subscribe event; conn_handle=1 attr_handle=10 reason=1 nrevn=0 currn=0 pre
i=1
ble_svc_gatt_changed_val_handle=10
data=ssid 4 Len=6 step 0
att=73 73 69 64 20 34
ble ssid_len = 4
data=asdf Len=4 step 1
att=61 73 64 66
ble ssid = asdf
data=pwd 8 Len=5 step 2
att=70 77 64 20 38
ble pwd_len = 8
data=12345678 Len=8 step 3
att=31 32 33 34 35 36 37 38
ble pwd = asdf
connection, notify success
ble smart cfg connect ap! ssid:asdf pwd:12345678
ble smart config end...
ble smart config ssid pwd trans Done(0xFE) ssid asdf pwd 12345678
start CMD:/tmp/connect_ap.sh asdf 12345678

```

手机通过writte发送数据

- 1.ssid 长度//ssid长度
- 2.ssid //ssid
- 3.pwd 长度//密码长度
- 4.pwd //pwd

通过iwpriv命令查询状态
成功过后，发送成功给手
机端

5 移植自定义应用代码与退出 ble 线程

5.1 客户应用代码建议在我们基础上修改

1. 在 ble_smart_cfg.h 头文件中屏蔽该宏#define ATBM_BLE_SMART_APP 1

2. 在 ble_host\nimble_v42\apps\main.c 中把 main 函数入口改名。

5.2 退出 ble 线程方式

1. 把 hif_ioctl_loop() 屏蔽，调用 atbm_ble_exit() 退出。在任意地方调用 atbm_ble_is_quit=1 就可以退出 ble 线程。



5.3 缩减 ble 代码大小

配置选项在\ble_host\os\linux_app\include\syscfg\syscfg.h

1. 关闭 bt_shell 测试代码把该宏置 MYNEWT_VAL_SHELL_CMD (0)



2.GAP 层关闭 Broadcaster 角色把该宏

MYNEWT_VAL_BLE_ROLE_BROADCASTER 置 0.

3.GAP 层关闭 Central 角色把该宏

MYNEWT_VAL_BLE_ROLE_CENTRAL 置 0.

4.GAP 层关闭 Observer 角色把该宏

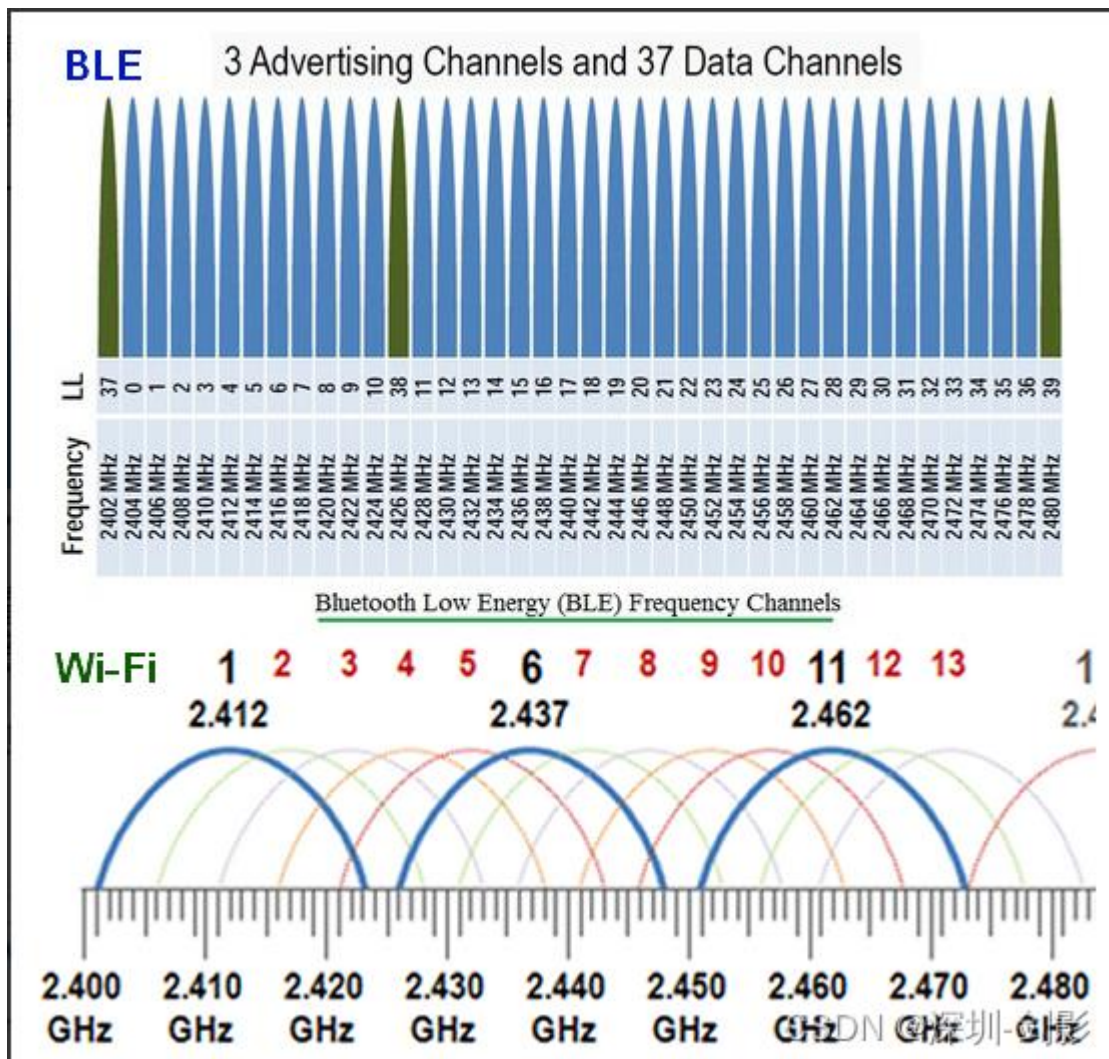
MYNEWT_VAL_BLE_ROLE_OBSERVER 置 0.

错误码信息

```
: #define BLE_HS_EAGAIN 1
: #define BLE_HS_EALREADY 2
: #define BLE_HS_EINVAL 3
: #define BLE_HS_EMMSGSIZE 4
: #define BLE_HS_ENOENT 5
: #define BLE_HS_ENOMEM 6
: #define BLE_HS_ENOTCONN 7
: #define BLE_HS_ENOTSUP 8
: #define BLE_HS_EAPP 9
: #define BLE_HS_EBADDATA 10
: #define BLE_HS_EOS 11
: #define BLE_HS_ECONTROLLER 12
: #define BLE_HS_ETIMEOUT 13
: #define BLE_HS_EDONE 14
: #define BLE_HS_EBUSY 15
: #define BLE_HS_EREJECT 16
: #define BLE_HS_EUNKNOWN 17
: #define BLE_HS_EROLE 18
: #define BLE_HS_ETIMEOUT_HCI 19
: #define BLE_HS_ENOMEM_EVT 20
: #define BLE_HS_ENOADDR 21
: #define BLE_HS_ENOTSYNCED 22
: #define BLE_HS_EAUTHEN 23
: #define BLE_HS_EAUTHOR 24
: #define BLE_HS_EENCRYPT 25
: #define BLE_HS_EENCRYPT_KEY_SZ 26
: #define BLE_HS_ESTORE_CAP 27
: #define BLE_HS_ESTORE_FAIL 28
: #define BLE_HS_EPREEMPTED 29
: #define BLE_HS_EDISABLED 30
:
```


1. BLE RF 测试指令

BLE 广播信道



需要把 BLE 协议栈运行起来才能测试

nimble_linux &

./atbm_cli btshell advertise stop

//停止 ble 广播，默认是启动广播

1. atbm_cli btshell dtm-tx

指令	atbm_cli btshell dtm-tx
格式	atbm_cli btshell dtm-tx chan=<channel> len=<length> payload=<packet_Payload> phyMode=<PHY Mode>
参数	<channel> : 信道 channel = (F-2402)/2, 用十六进制表示, 是 RF channel 不是 Channel index 比如 channel=2 反推 RF channel 为 2406 公式: (2406-2402)/2=2 <length> : Length in bytes of payload data in each packet (0x00-0xff) <packet_Payload>: 0x00 PRBS9 0x01 Repeated '11110000' 0x02 Repeated '10101010' 0x03 PRBS15

	0x04 Repeated '11111111' 0x05 Repeated '00000000' 0x06 Repeated '00001111' 0x07 Repeated '01010101' <PHY Mode>: 1 1M PHY 2 2M PHY 3 S8 Code PHY 4 S2 Code PHY
功 能 描述	开始 Direct Test TX Mode
返 回 值	OK or Fail
指 令 示例	atbm_cli btshell dtm-tx chan=2 len=27 payload=1 phyMode=1

2. atbm_cli btshell dtm-tx stop

指令	atbm_cli btshell dtm-tx stop
格式	atbm_cli btshell dtm-tx stop
参数	无
功能描述	结束 Direct Test TX Mode
返回值	OK or Fail
指令示例	atbm_cli btshell dtm-tx stop

3. atbm_cli btshell dtm-rx

指令	atbm_cli btshell dtm-rx
格式	atbm_cli btshell dtm-rx chan=<channel> phyMode=<PHY Mode>
参数	<channel> : 信道 channel = (F-2402)/2, 用十六进制表示, 是 RF channel 不是 Channel index 比如 channel=2 反推 RF channel 为 2406 公式: (2406-2402)/2=2 <PHY Mode>: 1 1M PHY 2 2M PHY 3 S8 Code PHY 4 S2 Code PHY
功能描述	开始 Direct Test RX Mode
返回值	OK or Fail
指令示例	atbm_cli btshell dtm-rx chan=2 phyMode=1

4. atbm_cli btshell dtm-rx stop

指令	atbm_cli btshell dtm-rx stop
格式	atbm_cli btshell dtm-rx stop
参数	无
功能描述	结束 Direct Test RX Mode

返回值	OK or Fail
指令示例	atbm_cli btshell dtm-rx stop

频率要求大体如下:

1Msym/s

当使用 00001111 测试序列时, 平均频率偏差应符合 $225\text{ kHz} \leq \Delta f_{1\text{avg}} \leq 275\text{ kHz}$ 的范围

当使用 10101010 测试序列时, 至少 99.9%的最大频率偏差应大于等于 185Khz

2Msym/s

当使用 00001111 测试序列时, 平均频率偏差应符合 $450\text{ kHz} \leq \Delta f_{1\text{avg}} \leq 550\text{ kHz}$ 的范围

当使用 10101010 测试序列时, 至少 99.9%的最大频率偏差应大于等于 370Khz

BLE 标准对接收灵敏度的规定是这样的:

最大载荷包长度 (字节)	比特误码率 (%)
1 to 37	0.1
38 to 63	0.064
64 to 127	0.034
128 to 255	0.017



CONTACT INFORMATION

AltoBeam (China) Inc.

Address: B808, Tsinghua Tongfang Hi-Tech Plaza, Haidian, Beijing, China 100083

Tel: (8610) 6270 1811

Fax: (8610) 6270 1830

Website: www.altobeam.com

Email: support@altobeam.com

DISCLAIMER

Information in this document is provided in connection with AltoBeam products. No license, express or implied, by estoppels or otherwise, to any intellectual property rights is granted by this document. Except as provided in AltoBeam's terms and conditions of sale for such products, AltoBeam assumes no liability whatsoever, and AltoBeam disclaims any express or implied warranty, relating to sale and/or use of AltoBeam products including liability or warranties relating to fitness for a particular purpose, merchantability, or infringement of any patent, copyright or other intellectual property right.

AltoBeam may make changes to specifications and product descriptions at any time, without notice.

Designers must not rely on the absence or characteristics of any features or instructions marked "reserved" or "undefined." AltoBeam reserves these for future definition and shall have no responsibility whatsoever for conflicts or incompatibilities arising from future changes to them.

Unauthorized use of information contained herein, disclosure or distribution to any third party without written permission of AltoBeam is prohibited.

AltoBeam™ is the trademark of AltoBeam. All other trademarks and product names are properties of their respective owners.

Copyright © 2007~2020 AltoBeam, all rights reserved